# Event Storming 1 · Criação da Ordem de Serviço

Da chegada do cliente até o envio do orçamento para aprovação. Cada post-it usa o nome real do código (comando, rota, agregado, evento). A ligação completa com handlers e testes está em `docs/ddd/rastreabilidade.md`.

Board no Miro: https://miro.com/app/board/uXjVEVRxgE4=/. O conteúdo é o mesmo deste PDF; a fonte é `docs/ddd/fontes/`.

**Ator** quem dispara

**Comando** intenção / ação (rota)

**Agregado** quem decide e guarda o estado

**Evento** fato que aconteceu

**Política** sempre que… então…

**Regra de negócio** invariante verificada

**Read model** consulta usada na decisão

**Hot spot** limitação / próximo passo

tempo →

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Ator** | Atendente / Admin | Atendente / Admin | Atendente / Admin | Mecânico / Admin | Mecânico / Admin | Mecânico / Admin | Atendente / Admin |
| **Read model** | **Buscar cliente por CPF/CNPJ**<br>`SearchCustomerByDocumentQuery`<br>`POST /api/admin/customers/search` | | | | **Catálogo de serviços**<br>`ListServicesQuery`<br>`GET /api/admin/services` | **Peças e insumos com saldo**<br>`ListPartsQuery`<br>`GET /api/admin/parts` | **Detalhe da O.S.**<br>`GetServiceOrderByIdQuery` |
| **Comando** | **Cadastrar cliente**<br>`CreateCustomerCommand`<br>`POST /api/admin/customers` | **Cadastrar veículo**<br>`CreateVehicleCommand`<br>`POST /api/admin/vehicles` | **Abrir O.S.**<br>(CPF/CNPJ + placa + relato)`OpenServiceOrderCommand`<br>`POST /api/admin/service-orders` | **Iniciar diagnóstico**<br>`StartDiagnosisCommand`<br>`POST …/{id}/diagnosis/start` | **Incluir serviço**<br>`AddServiceItemCommand`<br>`POST …/{id}/services`<br>(remover: `RemoveItemCommand`) | **Incluir peça ou insumo**<br>`AddPartItemCommand`<br>`POST …/{id}/parts`<br>(remover: `RemoveItemCommand`) | **Enviar orçamento**<br>`SendQuoteCommand`<br>`POST …/{id}/quote/send` |
| **Agregado** | **Customer**<br>`Document`, `Phone`, `Email`, `Address` | **Vehicle**<br>`LicensePlate`, `VehicleYear` | **ServiceOrder**<br>`ServiceOrder.Open` | **ServiceOrder**<br>`StartDiagnosis` | **ServiceOrder**<br>→ `ServiceItemAddServiceItem` | **ServiceOrder**<br>→ `PartItemAddPartItem` | **ServiceOrder**<br>→ `QuoteSendQuote` |
| **Evento** | **Cliente cadastrado**<br>(fato no agregado, sem evento de domínio) | **Veículo cadastrado**<br>(fato no agregado, sem evento de domínio) | **O.S. aberta**<br>— status<br>**Recebida**<br>`ServiceOrderOpened` | **Diagnóstico iniciado**<br>— status<br>**Em diagnóstico**<br>(registro em `StatusHistory`) | **Serviço incluído na O.S.**<br>(nome e preço copiados do catálogo) | **Peça/insumo incluído na O.S.**<br>(nome e preço copiados do estoque) | **Orçamento enviado**<br>— status<br>**Aguardando aprovação**<br>`QuoteSent` |
| **Política / regra** | CPF/CNPJ com dígitos verificadores; documento único (`ux_customers_document`) | Placa antiga (`ABC1234`) ou Mercosul (`ABC1D23`), única; ano ≥ 1900 e ≤ ano seguinte; cliente precisa existir | **ServiceOrderOpeningPolicy**<br>: veículo pertence ao cliente e não tem outra O.S. aberta (índice único filtrado) | Só sai de **Recebida**<br>; senão `serviceOrder.invalidTransition` (409) | Só em **Em diagnóstico**<br>; serviço ativo; quantidade > 0 | Só em **Em diagnóstico**<br>; peça ativa e com saldo (`Part.HasAvailable`) | **Orçamento automático**<br>: total = serviços + peças; exige ≥ 1 item; gera token de aprovação (só o hash SHA-256 é guardado) |
| **Hot spot** | | | | | | | O orçamento é **disponibilizado** pelo token devolvido na resposta; envio automático por e-mail/WhatsApp não implementado |

**Eventos de domínio deste fluxo**

- `ServiceOrderOpened` — `ServiceOrder.Open`
- `QuoteSent` — `ServiceOrder.SendQuote`
- Os dois são gravados no Outbox na mesma transação do commit (`UnitOfWork` + `OutboxWriter`).

**Transação**

- Cada comando carrega o agregado, aplica a regra e faz um único commit em transação (`UnitOfWork.CommitAsync`).
- `ServiceOrder` tem `rowversion`: alterações concorrentes geram 409 (`common.concurrencyConflict`).

# Event Storming 2 · Acompanhamento da Ordem de Serviço

Da decisão do cliente sobre o orçamento até a entrega do veículo, mais as consultas do cliente e da administração.

tempo →

| | | | | | | |
|---|---|---|---|---|---|---|
| **Ator** | Cliente (link) ou Atendente / Admin (balcão) | Sistema | Cliente (link) ou Atendente / Admin (balcão) | Mecânico / Admin | Atendente / Admin | Cliente (anônimo) |
| **Comando** | **Aprovar orçamento**<br>`ApproveQuoteByTokenCommand`<br>`POST /api/public/quotes/{token}/approve`<br>`ApproveQuoteCommand`<br>`POST …/{id}/quote/approve` | **Baixar estoque**<br>(na mesma transação da aprovação)`Part.DeductForServiceOrder` | **Recusar orçamento**<br>(com motivo)`RejectQuoteByTokenCommand`<br>`POST /api/public/quotes/{token}/reject`<br>`RejectQuoteCommand`<br>`POST …/{id}/quote/reject` | **Finalizar O.S.**<br>`CompleteServiceOrderCommand`<br>`POST …/{id}/complete` | **Entregar veículo**<br>`DeliverServiceOrderCommand`<br>`POST …/{id}/deliver` | **Consultar andamento**<br>`LookupServiceOrderQuery`<br>`POST /api/public/service-orders/lookup`<br>(número + CPF/CNPJ no corpo) |
| **Agregado** | **ServiceOrder**<br>→ `QuoteApproveQuote` | **Part**<br>→ `StockMovement` | **ServiceOrder**<br>→ `QuoteRejectQuote` | **ServiceOrder**<br>`Complete` | **ServiceOrder**<br>`Deliver` | `PublicServiceOrderDto`: status, itens, orçamento atual |
| **Evento** | **Orçamento aprovado**<br>— status<br>**Em execução**<br>; grava `ExecutionStartedAtQuoteApproved` | **Estoque baixado**<br>`StockMovementType.Deduction`, motivo `serviceOrder:{número}` | **Orçamento recusado**<br>— volta para<br>**Em diagnóstico**<br>`QuoteRejected` | **O.S. finalizada**<br>— status<br>**Finalizada**<br>; grava `CompletedAtServiceOrderCompleted` | **Veículo entregue**<br>— status<br>**Entregue**<br>; grava `DeliveredAt`; O.S. encerrada | |
| **Política / regra** | **QuoteApprovalPolicy**<br>: sempre que o orçamento é aprovado, baixa todas as peças da O.S.; sem saldo → 409 e rollback de tudo. O token é invalidado. | Saldo nunca fica negativo (`inventory.insufficientStock`); `Part` tem `rowversion` | Motivo obrigatório; o envio seguinte cria nova versão do `Quote` (`Version + 1`) | **NotifyCustomerOnServiceOrderCompleted**<br>: sempre que a O.S. é finalizada, avisa o cliente (pelo Outbox → `LogNotifier`) | Só a partir de **Finalizada**<br>; depois da entrega o veículo pode abrir nova O.S. | Número e documento precisam ser do mesmo cliente; senão 404 (não revela se a O.S. existe) |
| **Hot spot** | | | | Aviso ao cliente só registrado em log; integração real (e-mail/WhatsApp) é próximo passo | | |

## Máquina de status (regras de transição em `ServiceOrder`)

| | | | | | | |
|---|---|---|---|---|---|---|
| **Status** | **Recebida**<br>`Received` | **Em diagnóstico**<br>`InDiagnosis` | **Aguardando aprovação**<br>`WaitingApproval` | **Em execução**<br>`InProgress` | **Finalizada**<br>`Completed` | **Entregue**<br>`Delivered` |
| **Ação que leva** | `Open` | `StartDiagnosis` OU `RejectQuote` | `SendQuote` | `ApproveQuote` | `Complete` | `Deliver` |

### Consultas da administração (read models)

- `ListServiceOrdersQuery` — `GET /api/admin/service-orders` (paginada, filtro por status e placa)
- `GetServiceOrderByIdQuery` — `GET /api/admin/service-orders/{id}` (itens, orçamentos, histórico de status)
- `GetAverageExecutionTimeQuery` — `GET /api/admin/metrics/average-execution-time?from&to` (média de `CompletedAt – ExecutionStartedAt`)

### Garantias

- Toda transição grava `StatusHistory` (status anterior, novo, usuário, nota e data).
- Ação fora de ordem → `serviceOrder.invalidTransition` (409).
- Eventos vão para o Outbox na mesma transação; o `OutboxProcessor` entrega depois do commit.

# Event Storming 3 · Gestão de peças e insumos

Cadastro, entrada de estoque, uso na O.S. e baixa na aprovação do orçamento. Peça e insumo são o mesmo agregado `Part`, diferenciados por `PartType`.

tempo →

| | | | | | | |
|---|---|---|---|---|---|---|
| **Ator** | Admin | Admin | Admin | Mecânico / Admin | Sistema (na aprovação) | Equipe autenticada |
| **Comando** | **Cadastrar peça ou insumo**<br>`CreatePartCommand`<br>`POST /api/admin/parts` | **Alterar peça**<br>(nome, tipo, unidade, preço, mínimo)`UpdatePartCommand`<br>`PUT /api/admin/parts/{id}` | **Registrar entrada**<br>`RegisterStockEntryCommand`<br>`POST /api/admin/parts/{id}/stock-entries` | **Incluir peça na O.S.**<br>`AddPartItemCommand` | **Baixar estoque**<br>`QuoteApprovalPolicy` → `Part.DeductForServiceOrder` | **Consultar estoque**<br>`ListPartsQuery (belowMinimum=true)`<br>`GetPartByIdQuery` (com movimentações) |
| **Agregado** | **Part**<br>`PartCode`, `PartType { Part, Supply }`, `Money` | **Part**<br>`ChangeDetails` | **Part**<br>→ `StockMovementRegisterEntry` | **ServiceOrder**<br>(consulta `Part.HasAvailable`) | **Part**<br>→ `StockMovement` | `PartDto` / `PartDetailDto` / `StockMovementDto` |
| **Evento** | **Peça cadastrada**<br>(saldo inicial 0) | **Peça alterada** | **Entrada registrada**<br>`StockMovementType.Entry` com saldo após | **Peça/insumo incluído na O.S.** | **Estoque baixado**<br>`StockMovementType.Deduction` ligado à O.S. | |
| **Política / regra** | Código `A-Z 0-9 . -` até 30 e único (`ux_parts_code`); nome 2–120; estoque mínimo ≥ 0 | Desativar / reativar (`DeactivatePartCommand`, `ReactivatePartCommand`): inativa não entra em O.S. | Quantidade > 0 com até 3 casas (`Quantity`); motivo obrigatório | Só com saldo suficiente (`inventory.partUnavailable`, 422) | **QuoteApprovalPolicy**<br>: baixa agrupada por peça, na mesma transação da aprovação; falta de saldo desfaz tudo | `Part.IsBelowMinimum` = saldo < mínimo |
| **Hot spot** | | | | Saldo é conferido na inclusão e de novo na aprovação: entre os dois momentos outra O.S. pode consumir a peça (aprovação devolve 409) | | Estoque abaixo do mínimo só aparece na consulta; não há alerta automático nem pedido de compra |

### Movimentações

- `StockMovementType.Entry` — entrada manual com motivo.
- `StockMovementType.Deduction` — baixa por O.S. aprovada (`ServiceOrderId` preenchido).
- Cada movimentação guarda quantidade, saldo após, motivo e usuário.

### Concorrência

- `Part` tem `rowversion`; duas baixas simultâneas na mesma peça resultam em 409 para a segunda.
- Teste: `PublicServiceOrdersApiTests.ApproveByToken_InsufficientStock_Returns409AndRollsBack`.